



GITOPS CI/CD PIPELINE WITH ARGOCD

By Ahmed Tetteh | September 2026

```
king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps/devops-pipeline-manifests$ kubectl get pods -n production
NAME                                READY   STATUS    RESTARTS   AGE
devops-app-57d6f895bd-56pg9        1/1     Running   0           116s
devops-app-57d6f895bd-fvj2h        1/1     Running   0           116s
```

Building a Production-Grade GitOps CI/CD Pipeline

Project vision and DevOps significance

In this project, I'm building a fully automated CI/CD pipeline that is automatically triggered when a code has been pushed to the repository. This demonstrates production-ready DevOps skills because repetitive tasks are now handled automatically by the pipeline, freeing the engineer to handle other tasks.

Setting Up the GitOps Repository Structure

Environment and repository setup

In this step, I'm setting up my GitOps two-repo pattern structure to isolate my application source code from the deployment configuration.

```
king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps$ git clone https://github.com/kingswanzy2020/devops-  
pipeline-app.git  
git clone https://github.com/kingswanzy2020/devops-pipeline-manifests.git  
Cloning into 'devops-pipeline-app'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Compressing objects: 100% (2/2), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.  
Cloning into 'devops-pipeline-manifests'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Compressing objects: 100% (2/2), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.  
king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps$ k cluster-info  
Kubernetes control plane is running at https://127.0.0.1:35809  
CoreDNS is running at https://127.0.0.1:35809/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```


Containerizing a Database-Backed Flask API

Application and Docker build goals

In this step, I'm building the Flask REST API with a PostgreSQL database connected to it. Then I will build the image using multi-stage docker build and perform a local test with docker compose.

```
king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps/devops-pipeline-app
● $ curl http://localhost:5000/health
{"database":"connected","status":"healthy"}
```

Multi-stage Docker build strategy

The multi-stage build separates the build stage from the final image, which means the final image comes in a small size, without any caches being saved in it from the pip command

Provisioning Kubernetes Infrastructure as Code with Terraform

Infrastructure provisioning goals

In this step, I'm setting up Terraform to provision my Kubernetes cluster for me, so that I can have a more repeatable way of deploying everything with a single command.

```
king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps/devops-pipeline-app  
$ k get pods -n database  
NAME                                READY   STATUS    RESTARTS   AGE  
postgres-9c99776b5-ctbp2           1/1     Running   0           3m13s
```

Why Infrastructure as Code matters

Infrastructure as Code gives me a well-versioned configuration that I can apply with a single command, making Kubernetes resource deployments easier to create and destroy.

Deploying via GitOps with ArgoCD

ArgoCD configuration goals

In this step, I'm configuring ArgoCD and setting up GitOps so that ArgoCD simply watches the GitHub repo I point it to as the source of truth to sync with the cluster.

Kubernetes manifests and database connectivity

I created a deployment and service resource for the Flask app that connects to the database using environment variables.



Understanding the OutOfSync state

The app is OutOfSync because the image has not yet been built in the future pushed to my DockerHub repository

Automating Builds and Deployments with Jenkins CI

CI pipeline setup goals

In this step, I'm setting up the Jenkins pipeline that will build and push the image to DockerHub when it detects a new a new commit to the app repo.

How Jenkins and ArgoCD communicate without direct API calls

Jenkins updates the image in the manifest.yml file in within the repo ArgoCD watches. So, ArgoCD in turn uses the repo as its single source of truth to make changes within the cluster. Jenkins does this by watching for specific changes within the Git repo, then it builds and pushes the image to docker hub.

Production Health Checks and ArgoCD Self-Healing

```
^Cking@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps/devops-pipeline-manifests$ argocd app get devops-app
apps  Deployment  production  devops-app  Synced  Healthy  deployment.apps/devops-app configured
Service  production  devops-app-service  Synced  Healthy
king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps/devops-pipeline-manifests$ kubectl get pods -n production -w
NAME                                READY  STATUS  RESTARTS  AGE
devops-app-57d6f895bd-gqcnf         1/1    Running  0          2m57s
devops-app-57d6f895bd-lck9r         1/1    Running  0          3m47s
```

Liveness vs. readiness probes explained

In this project extension, I learned that liveness probes check whether a pod is running or stuck, and readiness probes check whether a pod is actually ready to accept traffic.

Reflections and Takeaways

Key tools and concepts learned

The key tools I used include Git, GitHub, ArgoCD, Jenkins and Vs-code. Key concepts I learnt include how apps are deployed, scaled and reconciled in production without repetitive manual steps, reducing the deployment time.

Time and challenges

This project took me approximately 2 hours. The most challenging part was setting up the Jenkins pipeline for the CI process.

Personal learning goals

I did this project today to learn how DevOps teams operate in product to automate processes.